using `each`. The value of each entry is the value of the aggregation function.

The `order` and `limit` methods come into their own when using groups.

For example, the following returns the three states with the highest orders, sorted by the order amount:

```
result = Order.group(:state).
               order("max(amount) desc").
               limit(3)
```

This code is no longer database independent—to sort on the aggregated column, we had to use the SQLite syntax for the aggregation function (`max`, in this case).

Scopes

As these chains of method calls grow longer, making the chains themselves available for reuse becomes a concern. Once again, Rails delivers. An Active Record *scope* can be associated with a Proc and therefore may have arguments:

```
class Order < ActiveRecord::Base
  scope :last_n_days, ->(days) { where('updated < ?' , days) }
end
```

Such a named scope would make finding the worth of last week's orders a snap.

```
orders = Order.last_n_days(7)
```

Simpler scopes may have no parameters at all:

```
class Order < ActiveRecord::Base
  scope :checks, -> { where(pay_type: :check) }
end
```

Scopes can also be combined. Finding the last week's worth of orders that were paid by check is just as straightforward: